

ASSESSMENT TOOL 1:

Constraint-Based Problem Solving

1. REAL-TIME FACE RECOGNITION SYSTEM – EDGE DEPLOYMENT

1.1 Problem Statement

A company wants to deploy a face recognition system on low-power edge devices such as Raspberry Pi. The system must process live video streams in real time, work under varying lighting conditions, use limited memory and CPU resources, and detect and recognize faces from a predefined dataset.

1.2 Objective

The objective is to design a lightweight face recognition system that can continuously capture video, detect faces, extract useful facial features, recognize registered individuals, and display the recognition result with minimum processing delay.

1.3 Constraints

The system must satisfy the following constraints:

- Real-time processing of live video.
- Low processing delay.
- Robustness to different lighting conditions.
- Low CPU and memory consumption.
- Recognition of faces belonging to a predefined dataset.
- Reduction of false recognition.

1.4 Proposed Computer Vision Pipeline

Image Acquisition → Preprocessing → Face Detection → Face Region Extraction → Feature Extraction → Face Recognition → Threshold Validation → Output

1.5 Image Acquisition

The system obtains a continuous video stream from a webcam or camera connected to the edge device.

OpenCV's `cv2.VideoCapture()` can be used to capture video frames.

The camera resolution should be selected according to the processing capability of the device. A moderate resolution such as **640 × 480** can provide sufficient facial information while reducing computational requirements.

1.6 Preprocessing

Preprocessing improves the quality of the captured frame before face detection.

The following operations can be performed:

- Resize the frame to reduce computation.
- Convert the frame from BGR to grayscale.
- Apply histogram equalization or CLAHE to improve contrast.
- Apply Gaussian filtering when noise is present.

Grayscale conversion reduces the amount of data because face detection does not always require three color channels.

1.7 Face Detection

After preprocessing, the system identifies the locations of faces.

An OpenCV Haar Cascade classifier can be used for lightweight face detection. The detector returns bounding boxes around detected faces.

Each detected face can be represented as:

(x, y, width, height)

Only these face regions are passed to the recognition stage, reducing unnecessary processing.

1.8 Feature Extraction

The detected face is cropped and resized to a standard size.

The facial image is normalized to reduce the effects of illumination and size differences.

For a lightweight OpenCV implementation, **Local Binary Patterns Histogram (LBPH)** can be used. LBPH describes local texture patterns in the face and is relatively suitable for CPU-limited environments.

1.9 Face Recognition

The recognition model is trained using images of registered individuals.

During operation:

1. A face is detected.
2. Facial features are extracted.
3. The features are compared with the trained dataset.
4. A similarity/distance score is produced.
5. The system decides whether the face belongs to a registered user.

If the similarity does not satisfy the selected threshold, the person is classified as **Unknown**.

1.10 Output

The system displays:

- Detected face bounding box.
- Recognized person's name.
- Recognition status.
- Unknown label when recognition confidence is insufficient.

OpenCV drawing functions such as `cv2.rectangle()` and `cv2.putText()` can be used for visualization.

1.11 Constraint Justification

Constraint	Solution
Real-time processing	Reduced resolution and lightweight algorithms
Varying lighting	Grayscale conversion and contrast enhancement
Limited CPU	Lightweight face detection and LBPH
Limited memory	Compact model and frame-by-frame processing
Predefined dataset	Train the recognizer using registered users
False recognition	Apply a suitable recognition threshold

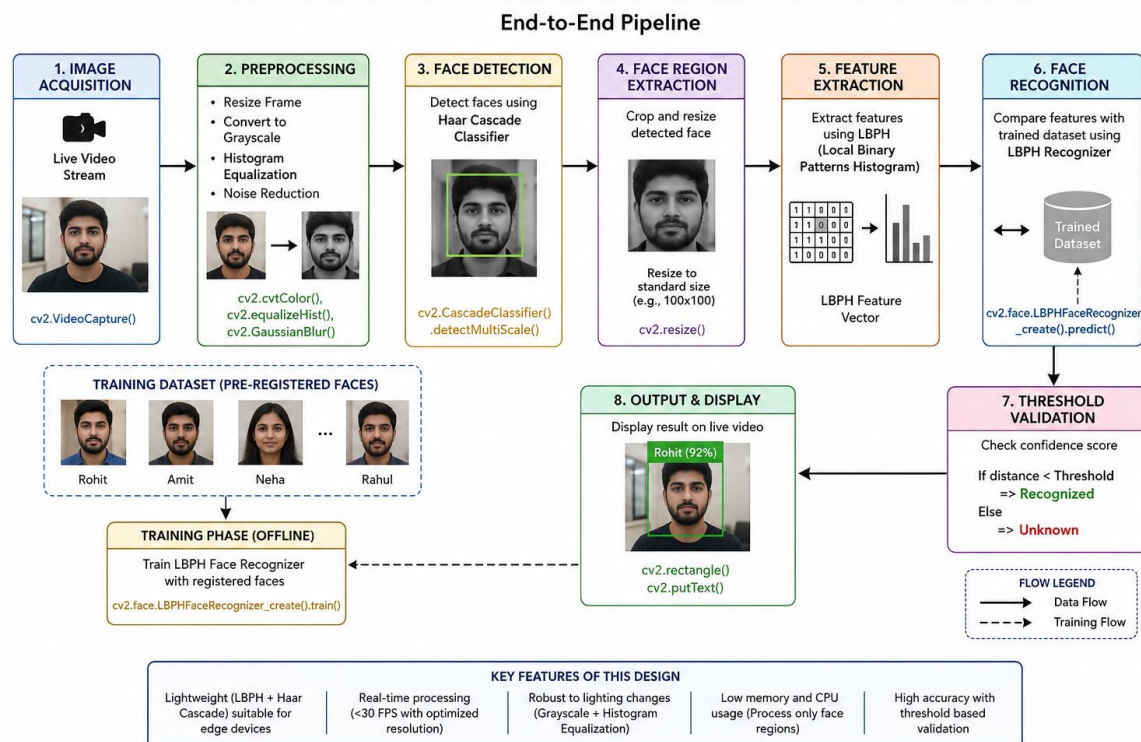
1.12 Expected Result

The system continuously captures video, detects faces, recognizes registered individuals, and displays their names in real time. Faces that do not meet the recognition threshold are classified as unknown.

1.13 Conclusion

The proposed system provides a lightweight face recognition solution suitable for edge devices. By reducing image resolution, using efficient preprocessing, restricting recognition to detected face regions, and applying a lightweight recognition algorithm, the system can operate under limited CPU and memory resources while maintaining practical recognition performance.

1. REAL-TIME FACE RECOGNITION SYSTEM – EDGE DEPLOYMENT



2. AUTOMATED VIDEO SURVEILLANCE WITH ALERT SYSTEM

2.1 Problem Statement

A security firm needs a video surveillance system that detects motion and unusual activities, generates alerts in real time, works with standard CCTV video feeds, and minimizes false positives in dynamic environments.

2.2 Objective

The objective is to develop an OpenCV-based surveillance system that continuously monitors CCTV footage, detects significant motion, filters unwanted noise, analyzes moving regions, and generates alerts when suspicious activity is detected.

2.3 Constraints

The system must:

- Process CCTV video continuously.
- Detect significant movement.
- Generate alerts with low delay.
- Handle noise.
- Handle dynamic backgrounds.
- Minimize false positives.
- Maintain real-time performance.

2.4 Proposed Computer Vision Pipeline

CCTV Feed → Frame Acquisition → Preprocessing → Background Subtraction → Motion Detection → Noise Filtering → Contour Detection → Region Analysis → Activity Validation → Alert → Display

2.5 Video Acquisition

The CCTV feed is obtained using OpenCV's `cv2.VideoCapture()`.

The input may come from:

- Webcam
- CCTV camera
- Video file
- Supported network camera stream

Frames are processed continuously.

2.6 Preprocessing

Each frame is converted to grayscale to simplify motion analysis.

Gaussian blur can be applied using `cv2.GaussianBlur()` to reduce small image variations and camera noise.

Frames can also be resized to reduce computational requirements.

2.7 Background Subtraction

Background subtraction separates moving foreground objects from the relatively stable background.

OpenCV's:

```
cv2.createBackgroundSubtractorMOG2()
```

can be used to create an adaptive background model.

The current frame is compared with the background model, and regions that differ significantly are identified as foreground.

2.8 Motion Detection

The foreground mask is processed to identify significant motion.

Thresholding separates foreground pixels from background pixels.

Small changes caused by image noise or lighting variations should not immediately trigger an alert.

2.9 Noise Removal

Morphological operations are applied to clean the foreground mask.

- **Opening** removes small isolated noise.
- **Closing** fills small gaps.
- **Dilation** strengthens detected regions.

These operations improve the quality of motion detection.

2.10 Contour Detection

Contours are extracted using:

```
cv2.findContours()
```

The area of each contour is calculated.

Very small contours are discarded because they are likely to be caused by noise rather than meaningful movement.

2.11 Dynamic Background Handling

Dynamic environments may contain:

- Moving trees.
- Shadows.
- Changing illumination.
- Small background movements.

An adaptive background model helps accommodate gradual background changes.

Additional filtering can be performed using:

- Minimum contour area.

- Region of Interest (ROI).
- Consecutive-frame validation.
- Morphological operations.

An alert should preferably be generated only when significant movement persists for several frames.

2.12 Alert Generation

When the detected movement satisfies the predefined conditions, the system generates an alert.

The alert can contain:

- Camera identification.
- Time of detection.
- Motion status.
- Detected region.

2.13 Visualization

The detected moving region can be displayed using bounding rectangles.

OpenCV functions such as `cv2.rectangle()` and `cv2.putText()` can be used.

2.14 Constraint Justification

Constraint	Solution
Motion detection	Background subtraction
Real-time alerts	Continuous frame processing
CCTV compatibility	<code>cv2.VideoCapture()</code>
Noise handling	Gaussian blur and morphology
Dynamic background	Adaptive MOG2 model
False-positive reduction	Area threshold and ROI
Real-time operation	Reduced resolution and efficient processing

2.15 Expected Result

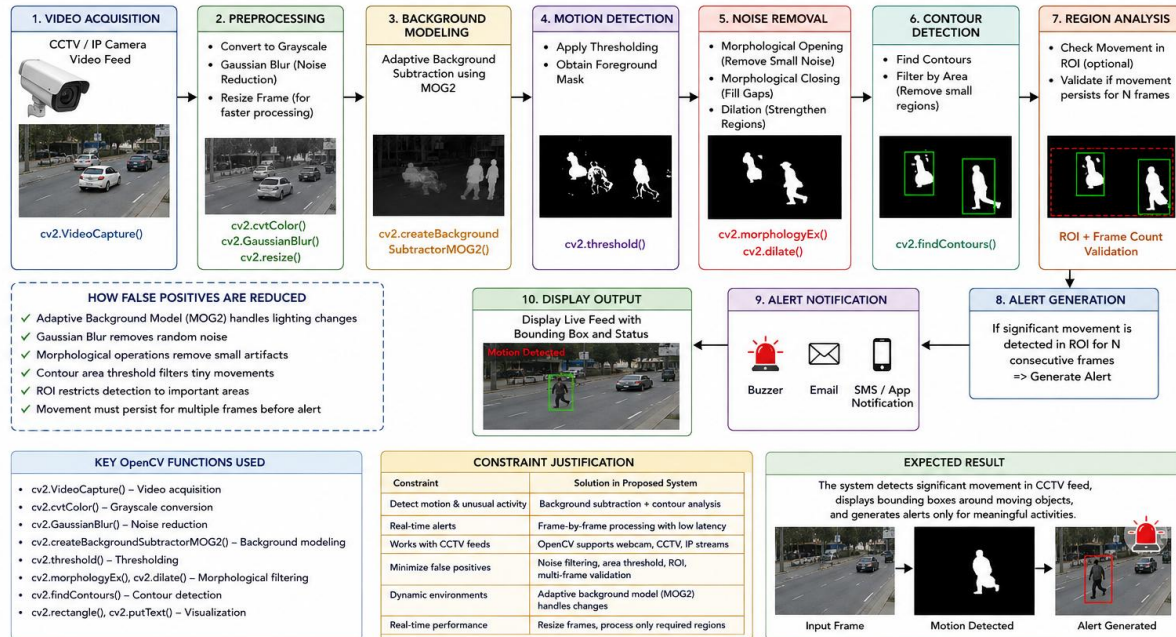
The system detects significant movement in CCTV footage and generates an alert only when the movement satisfies the defined conditions. Small noise and minor background changes are filtered out.

2.16 Conclusion

The proposed surveillance system combines background subtraction, morphological processing, contour analysis, and temporal validation to detect meaningful motion while reducing false alarms. The use of lightweight OpenCV operations allows the system to operate continuously with low processing delay.

2. AUTOMATED VIDEO SURVEILLANCE WITH ALERT SYSTEM

End-to-End OpenCV Based Pipeline



3. OCR SYSTEM FOR LOW-QUALITY DOCUMENTS

3.1 Problem Statement

A digitization company requires an OCR system that extracts text from low-resolution and noisy scanned documents, handles different fonts and illumination conditions, and processes documents efficiently in batches.

3.2 Objective

The objective is to develop an image preprocessing pipeline that improves poor-quality document images before sending them to an OCR engine.

3.3 Constraints

The system must handle:

- Low-resolution documents.
- Image noise.
- Uneven illumination.
- Different font styles.
- Poor character contrast.
- Large numbers of documents.
- Efficient batch processing.

3.4 Proposed Computer Vision Pipeline

Document Image → Resizing → Grayscale Conversion → Noise Removal → Contrast Enhancement → Thresholding → Morphological Processing → Text Segmentation → OCR → Text Output

3.5 Image Acquisition

The input document can be obtained from a scanner or camera.

OpenCV's `cv2.imread()` can be used to load document images.

3.6 Image Resizing

Low-resolution images can be enlarged before OCR.

`cv2.resize()` can be used with an appropriate interpolation method such as cubic interpolation.

Upscaling makes small characters more distinguishable to subsequent processing stages.

3.7 Grayscale Conversion

The image is converted into grayscale using:

`cv2.cvtColor()`

This reduces the image from three color channels to one intensity channel and decreases computational requirements.

3.8 Noise Removal

Low-quality documents may contain:

- Scanner noise.
- Dust.
- Small spots.
- Salt-and-pepper noise.

Median filtering using `cv2.medianBlur()` can reduce isolated noise while preserving character boundaries.

Gaussian filtering can also be used when appropriate.

3.9 Contrast Enhancement

Poor-quality documents may have weak contrast between text and background.

Histogram equalization or **CLAHE** can improve local contrast.

CLAHE is particularly useful when illumination varies across different parts of a document.

3.10 Thresholding

Thresholding separates text from the background.

Adaptive thresholding is useful when illumination is uneven.

OpenCV provides:

`cv2.adaptiveThreshold()`

It calculates thresholds locally rather than applying one threshold to the entire image.

3.11 Morphological Processing

Morphological operations improve the structure of the characters.

- **Opening:** removes small unwanted objects.
- **Closing:** fills small gaps.
- **Dilation:** strengthens weak character regions.
- **Erosion:** removes small unwanted connections.

The operation should be selected according to the document characteristics.

3.12 Text Segmentation

The processed document is divided into meaningful regions such as:

- Text blocks.
- Lines.
- Words.
- Characters.

Contours and projection-based techniques can help identify text regions.

3.13 OCR Processing

The preprocessed image is passed to an OCR engine such as Tesseract.

The OCR engine identifies characters and converts the visual text into machine-readable text.

OpenCV is mainly responsible for improving the input image before OCR.

3.14 Batch Processing

For processing large numbers of documents:

- Process documents sequentially.
- Avoid unnecessary color operations.
- Use suitable image resolution.
- Reuse preprocessing parameters.
- Process only required regions when possible.

3.15 Constraint Justification

Constraint	Solution
Low resolution	Image upscaling
Noise	Median/Gaussian filtering
Uneven illumination	CLAHE
Poor text-background contrast	Adaptive thresholding

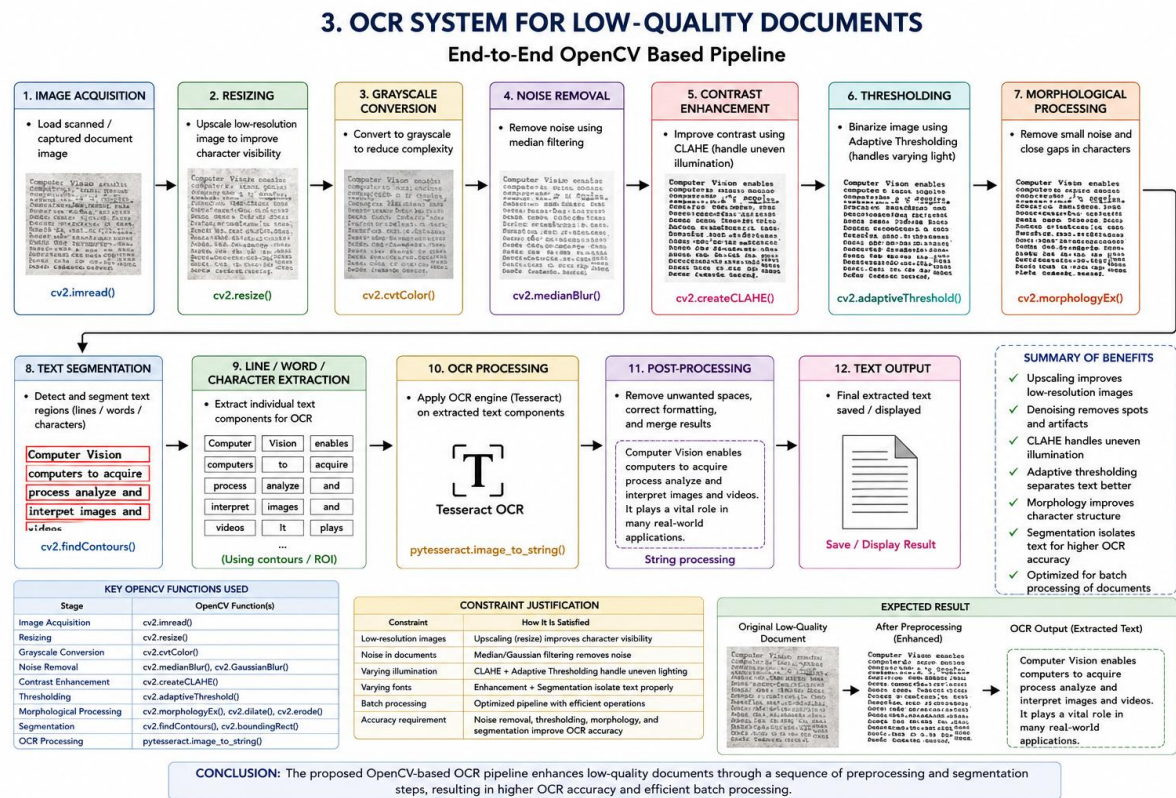
Constraint	Solution
Broken characters	Morphological processing
Different fonts	Proper segmentation and enhancement
Large document volume	Efficient batch processing

3.16 Expected Result

The system produces a cleaner, enhanced document image and extracts machine-readable text with improved OCR accuracy compared with processing the original low-quality image directly.

3.17 Conclusion

The proposed OCR pipeline improves document quality through resizing, denoising, contrast enhancement, adaptive thresholding, and morphological processing. These stages make characters clearer and provide a better input to the OCR engine while maintaining efficient batch processing.



6. REAL-TIME OBJECT DETECTION ON MOBILE DEVICE

6.1 Problem Statement

A mobile application must detect objects using the camera, operate in real time, and use limited CPU and memory resources.

6.2 Objective

The objective is to develop a lightweight OpenCV-based object detection pipeline that captures camera frames, identifies objects, filters incorrect detections, and displays the results with minimum delay.

6.3 Constraints

The system must:

- Use a mobile camera.
- Detect objects continuously.
- Provide real-time output.
- Use limited CPU.
- Use limited memory.
- Reduce false detections.

6.4 Proposed Computer Vision Pipeline

Camera → Frame Acquisition → Resizing → Preprocessing → Object Detection → Confidence Filtering → Non-Maximum Suppression → Visualization → Output

6.5 Image Acquisition

The mobile camera continuously captures frames.

Only the required resolution should be used because processing unnecessarily large images increases CPU and memory requirements.

6.6 Frame Resizing

Camera images may have high resolutions.

The frame is resized to a smaller detection resolution using:

`cv2.resize()`

For example, a high-resolution camera frame can be converted to a smaller working image.

This reduces the number of pixels processed by the detection model.

6.7 Preprocessing

The resized image is prepared for the detection model.

Typical preprocessing includes:

- Color conversion.
- Pixel scaling.

- Normalization.
- Creation of a neural-network input blob.

OpenCV provides:

`cv2.dnn.blobFromImage()`

for preparing an image for a DNN model.

6.8 Object Detection

A lightweight pretrained object detection model can be loaded using OpenCV's DNN module.

The model analyzes the image and identifies objects based on its training classes.

Possible objects include:

- Person.
- Car.
- Bicycle.
- Chair.
- Mobile phone.

The actual classes depend on the model and dataset used.

6.9 Confidence Filtering

The detection model produces a confidence score for each detected object.

Only detections above a selected confidence threshold are accepted.

This reduces false detections.

6.10 Non-Maximum Suppression

A single object may generate several overlapping bounding boxes.

Non-Maximum Suppression, or **NMS**, removes redundant boxes and keeps the strongest detection.

This improves the visual output and prevents multiple detections of the same object.

6.11 Visualization

The detected objects are displayed using:

- `cv2.rectangle()` – Draw bounding boxes.
- `cv2.putText()` – Display object names and confidence values.

The final output is shown on the mobile application display.

6.12 Mobile Optimization

To satisfy mobile constraints:

- Use a lightweight detection model.
- Reduce input resolution.

- Avoid unnecessary image copies.
- Process only required frames when appropriate.
- Use hardware acceleration when supported.
- Release resources properly.

6.13 Constraint Justification

Constraint Solution

Camera-based detection Continuous camera acquisition

Real-time performance Low-resolution input and lightweight model

Limited CPU Efficient OpenCV DNN processing

Limited memory Compact model

6.14 Expected Result

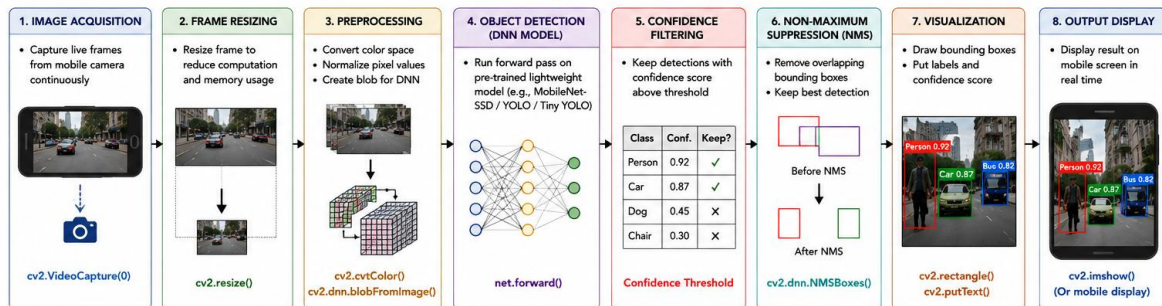
The mobile application detects objects from the live camera feed and displays bounding boxes, object labels, and confidence values with minimum processing delay.

6.15 Conclusion

A lightweight OpenCV DNN-based pipeline can provide real-time object detection on mobile devices. Reducing input resolution, selecting an efficient model, filtering detections, and avoiding unnecessary processing helps balance detection accuracy with CPU and memory limitations.

6. REAL-TIME OBJECT DETECTION ON MOBILE DEVICE

End-to-End OpenCV Based Pipeline



LIGHTWEIGHT DNN MODELS (SUITABLE FOR MOBILE)		KEY OPENCV FUNCTIONS USED		CONSTRAINT JUSTIFICATION		EXPECTED RESULT
	MobileNet-SSD	High speed, good accuracy, suitable for real-time apps		Constraint	How It Is Satisfied	Objects are detected from live camera feed in real time with bounding boxes, labels, and confidence scores.
	YOLOv5n / YOLOv8n	Very fast, efficient for mobile devices		Real-time detection	Lightweight DNN model + resized input + efficient pipeline	
	Tiny YOLO	Optimized for low computation and real-time detection		Limited CPU	Reduced resolution, optimized model, frame processing	
				Limited memory	Process one frame at a time, compact model	
				Accurate detection	Confidence filtering + NMS reduces false detections	
				Camera based	Continuous capture using <code>cv2.VideoCapture()</code>	
				Low latency	Optimized OpenCV DNN module and minimal operations	

OPTIMIZATION STRATEGIES FOR MOBILE DEVICES



Use low resolution (320x320 / 416x416) for faster processing



Use lightweight pre-trained models (SSD / YOLO Tiny etc.)



Process only necessary frames (e.g., skip frames if needed)



Use hardware acceleration (NEON / GPU / NPU) if available



Release unused resources and avoid memory leaks

APPLICATIONS

- Smart Camera Apps
- Surveillance
- Augmented Reality
- Robotics
- Assistive Applications

14. OCR FOR LICENSE PLATE RECOGNITION

14.1 Problem Statement

A license plate recognition system must detect vehicle license plates, extract the characters, and operate under varying lighting conditions.

14.2 Objective

The objective is to develop an OpenCV-based pipeline that identifies license plate regions from vehicle images or video, enhances the plate image, segments characters, and extracts the license number using OCR.

14.3 Constraints

The system must:

- Detect license plates.
- Extract plate characters.
- Handle different lighting conditions.
- Handle camera noise.
- Process multiple vehicles.
- Reduce incorrect detections.
- Provide results with low delay when used with video.

14.4 Proposed Computer Vision Pipeline

Camera/CCTV → Frame Acquisition → Resizing → Grayscale Conversion → Noise Removal → Edge Detection → Contour Detection → Plate Candidate Filtering → Plate Extraction → Illumination Enhancement → Thresholding → Character Segmentation → OCR → Validation → Output

14.5 Image Acquisition

Vehicle images can be obtained from CCTV cameras or roadside cameras.

For continuous video, OpenCV's `cv2.VideoCapture()` can be used.

14.6 Frame Preprocessing

The input frame can be resized to an appropriate working resolution.

The image is converted into grayscale using:

```
cv2.cvtColor()
```

This simplifies subsequent processing.

14.7 Noise Removal

Camera images can contain noise caused by:

- Low light.
- Compression.

- Camera sensors.
- Motion.

Gaussian or median filtering can be used to reduce unwanted noise.

14.8 License Plate Detection

License plates commonly have a rectangular structure with strong character edges.

Canny edge detection can be used to identify strong edges:

`cv2.Canny()`

The resulting edge image is processed to identify possible plate regions.

14.9 Contour Detection

Contours are extracted using:

`cv2.findContours()`

Candidate contours are filtered based on properties such as:

- Area.
- Width.
- Height.
- Aspect ratio.
- Rectangular shape.

This reduces the number of regions sent to the OCR stage.

14.10 Plate Extraction

Once a suitable plate candidate is found, the region is cropped from the original image.

Instead of processing the complete vehicle image, only the plate region is processed further.

This improves efficiency and focuses the OCR operation on relevant information.

14.11 Illumination Enhancement

License plates may be captured under:

- Bright sunlight.
- Shadows.
- Night conditions.
- Uneven lighting.
- Vehicle headlights.

Contrast enhancement techniques such as histogram equalization or **CLAHE** can improve the visibility of plate characters.

14.12 Thresholding

Adaptive thresholding can separate characters from the plate background when illumination is not uniform.

`cv2.adaptiveThreshold()` can be used.

This produces a binary representation suitable for character analysis.

14.13 Morphological Processing

Morphological operations can improve the character regions.

- Opening removes small noise.
- Closing fills small gaps.
- Dilation strengthens weak regions.
- Erosion removes unwanted small structures.

14.14 Character Segmentation

The plate is analyzed to identify individual characters.

Contours can be used to locate character candidates.

Character candidates can be filtered based on:

- Height.
- Width.
- Area.
- Aspect ratio.
- Position within the plate.

This helps eliminate unwanted regions.

14.15 OCR

The segmented plate image is passed to an OCR engine.

The OCR engine converts the visible characters into text.

The resulting text represents the estimated license plate number.

14.16 Validation

The extracted text can be checked against the expected license-plate format.

Validation helps reject outputs containing:

- Too few characters.
- Too many characters.
- Invalid character patterns.
- Unreasonable segmentation results.

14.17 Multiple Vehicle Handling

When multiple vehicles are present:

1. Detect candidate plate regions independently.
2. Process each plate region separately.
3. Assign each result to its corresponding vehicle.
4. Display the detected plate number.

This prevents the characters of different vehicles from being mixed.

14.18 Constraint Justification

Constraint	Solution
Plate detection	Canny edges and contour analysis
Text extraction	Character segmentation and OCR
Varying illumination	CLAHE and adaptive thresholding

14.19 Expected Result

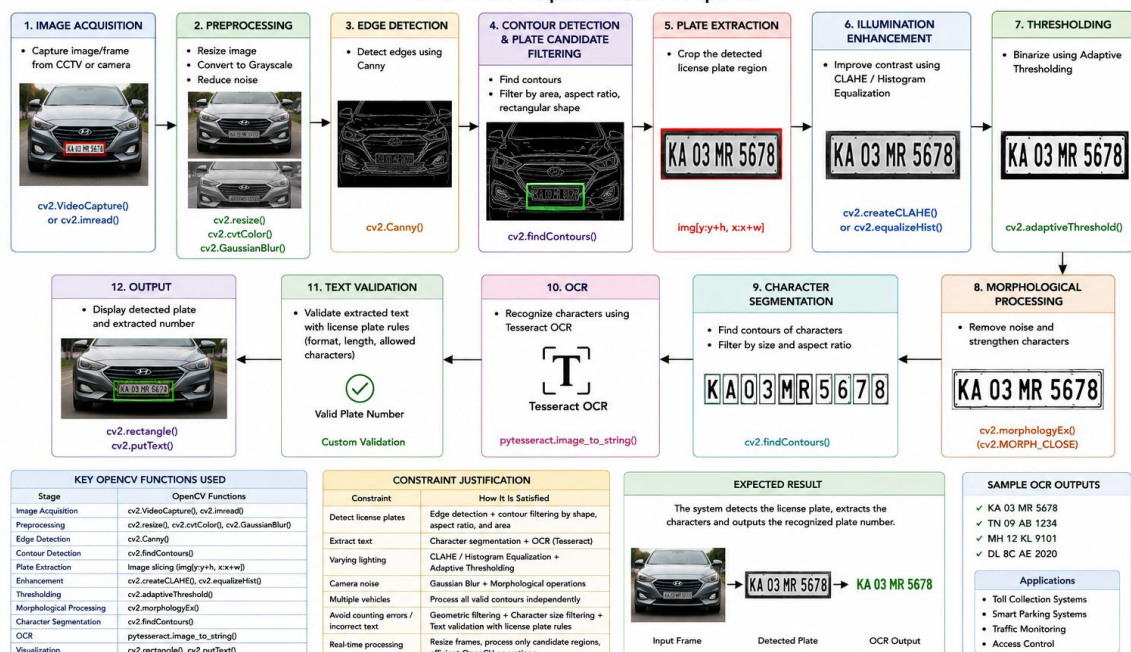
The system detects license plates from vehicle images or video, extracts the plate region, enhances the characters, performs OCR, validates the extracted text, and displays the estimated license plate number.

14.20 Conclusion

The proposed license plate recognition pipeline combines image preprocessing, edge detection, contour analysis, plate extraction, illumination correction, thresholding, character segmentation, and OCR. By processing only relevant plate regions and applying geometric and text validation, the system can improve accuracy while maintaining efficient operation.

14. OCR FOR LICENSE PLATE RECOGNITION

End-to-End OpenCV Based Pipeline



CONCLUSION: The proposed OpenCV-based pipeline uses a combination of edge detection, contour analysis, enhancement, segmentation, and OCR to accurately recognize license plates under different lighting and environmental conditions with high efficiency.